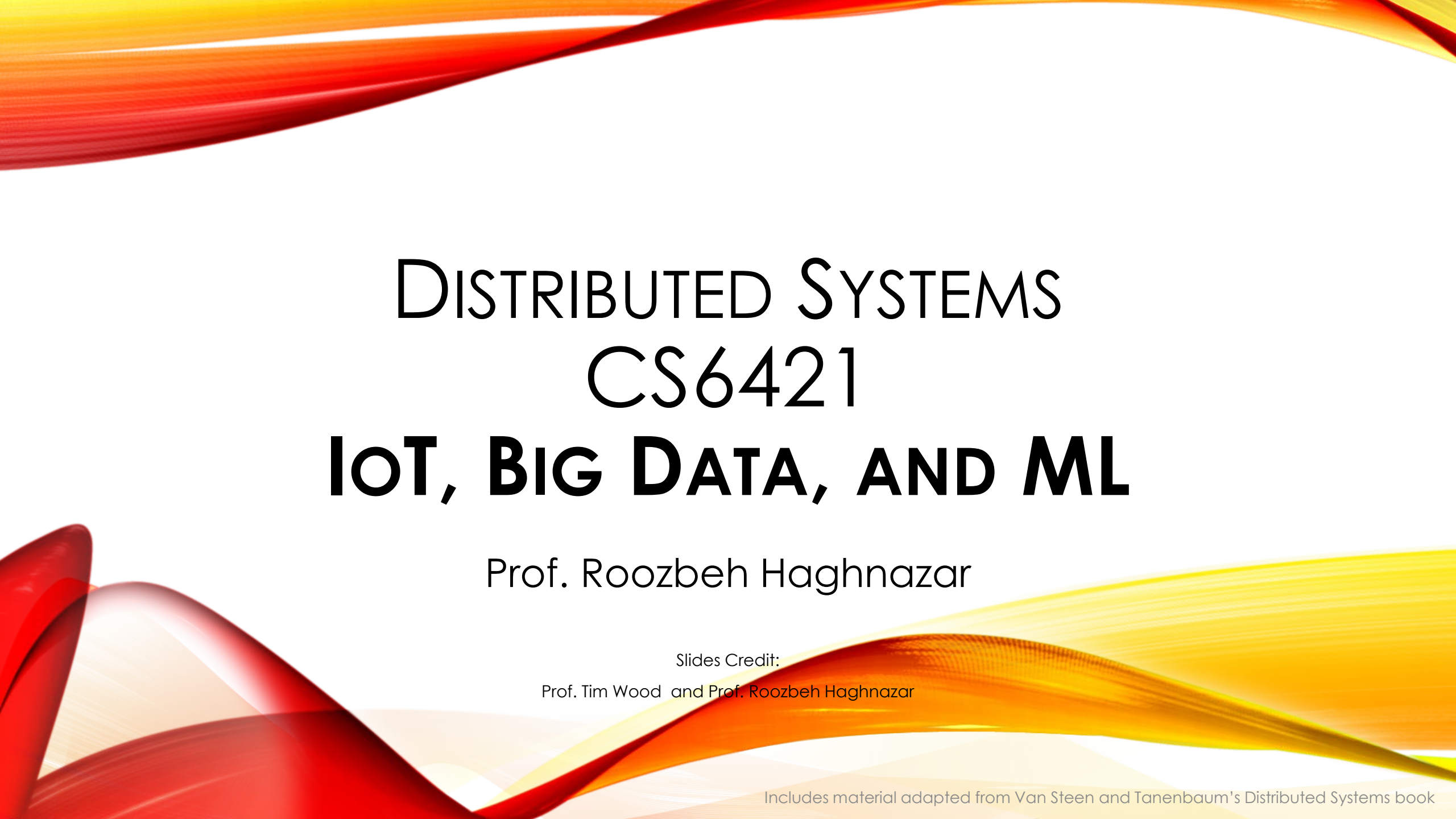




DISTRIBUTED SYSTEMS CS6421 **IoT, BIG DATA, AND ML**

Prof. Roozbeh Haghazadeh

Slides Credit:

Prof. Tim Wood and Prof. Roozbeh Haghazadeh

Includes material adapted from Van Steen and Tanenbaum's Distributed Systems book

FINAL PROJECT

Questions?

- Implementation phase!
 - Get coding!
 - Keep your code in GitHub
- See website for details on:
 - Report
 - ~15 minutes Video
- Timeline
 - Milestone 0: Form a Team
 - Milestone 1: Select a Topic
 - Milestone 2: Literature Survey
 - Milestone 3: Design Document
 - **Milestone 4: Final Video/Report**
 - **Bonus for submitting early!**

LAST TIME...

- Cloud Application Design
 - Monolith vs Microservices
 - Serverless
 - PaaS, IaaS, etc

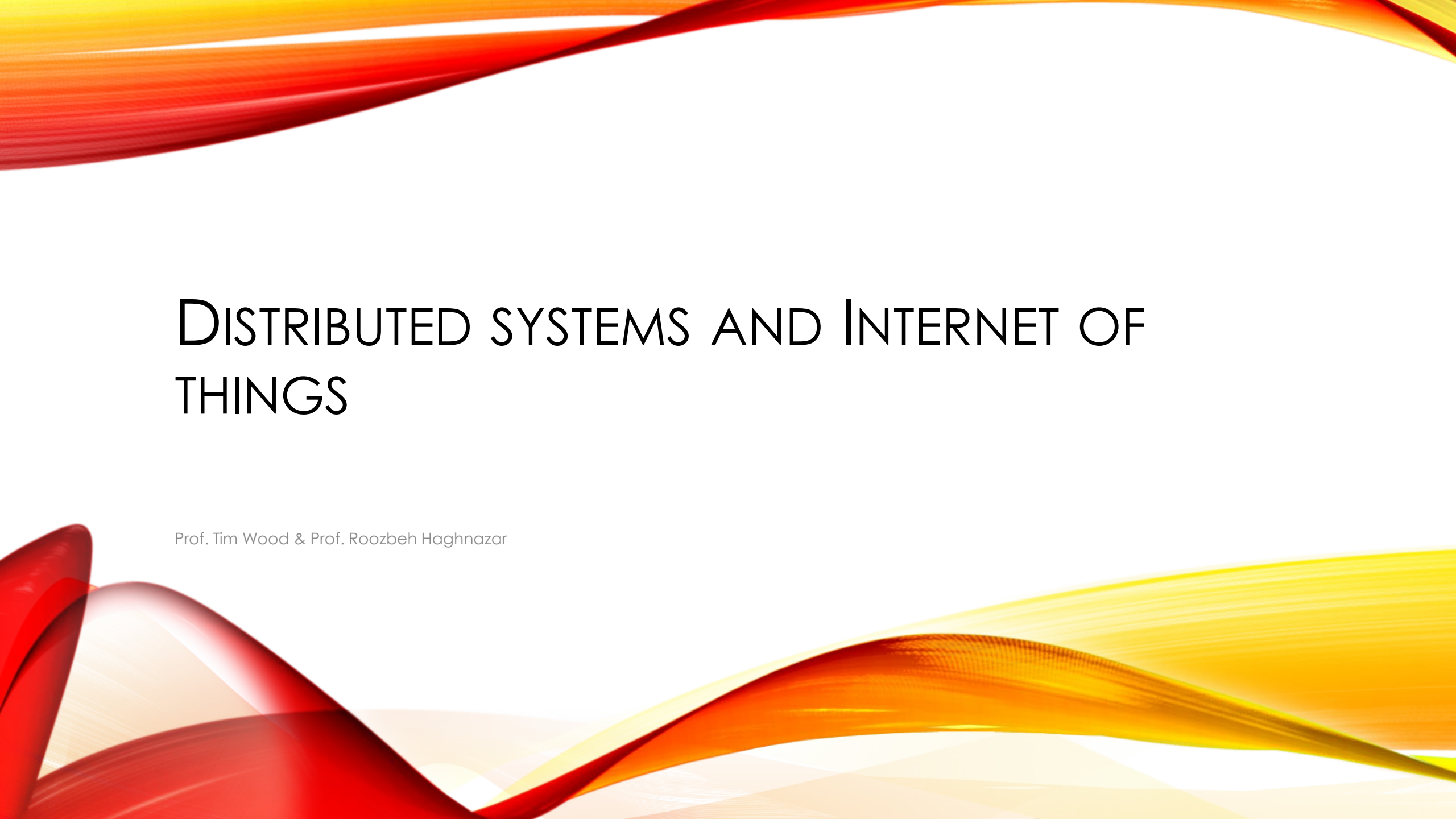
THIS TIME...

- IoT
- Big data
- Machine learning
- Security/Privacy
- Wrap up

DISTSYS CHALLENGES

- **Heterogeneity**
- **Openness**
- **Security**
- **Failure Handling**
- **Concurrency**
- **Quality of Service**
- **Scalability**
- **Transparency**

Covered Challenges



DISTRIBUTED SYSTEMS AND INTERNET OF THINGS

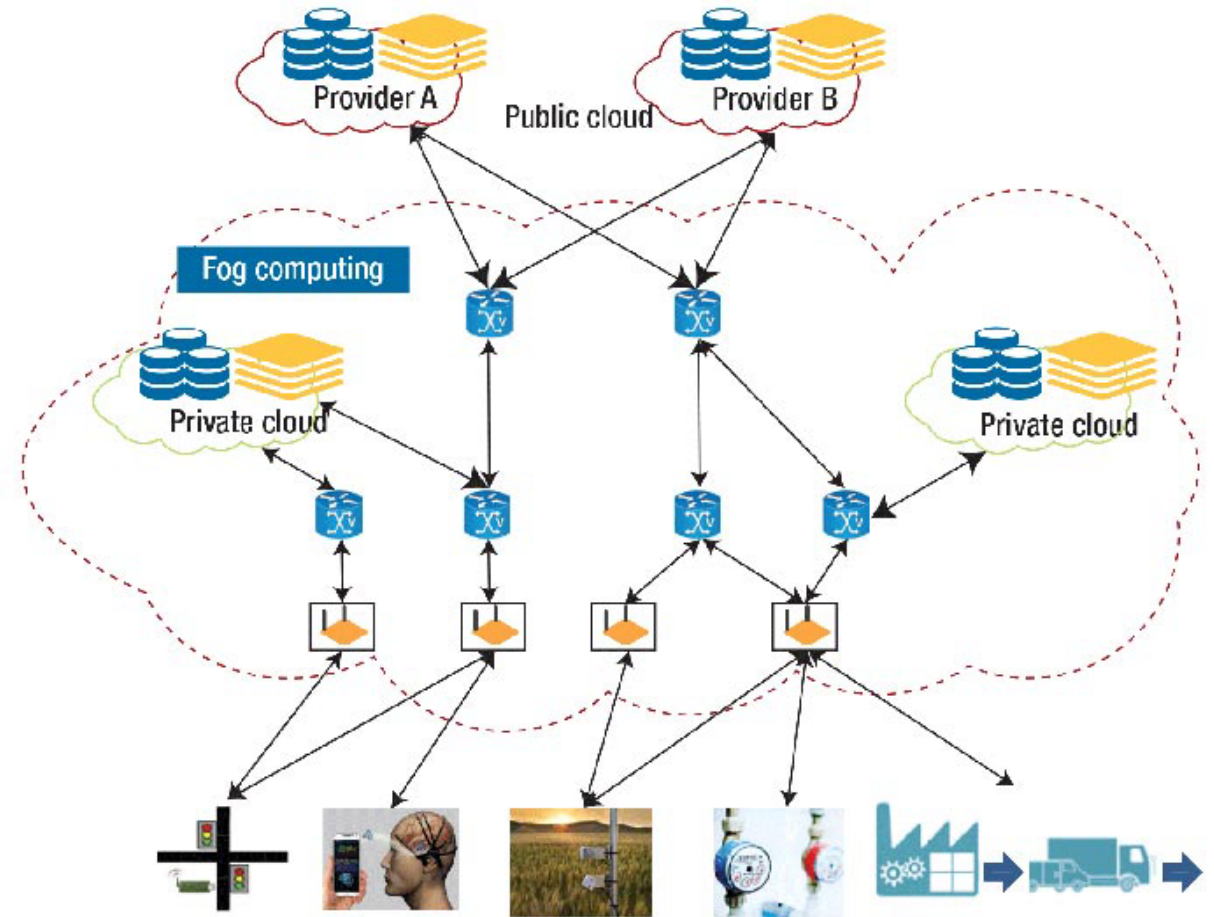
Prof. Tim Wood & Prof. Roozbeh Haghazadeh

WHAT IS IoT?

IoT is a collection of connected devices and services that work together to do useful stuff.

- Sensors
- Communication
- Computation
- Service

In IoT there is billions of smart objects, such as Sensors, Actuators, Smart phones and Smart Vehicles, etc. are connected with each other to sensing physical signals and giving better service in real time human need.



Edge → Fog → Cloud: The Modern Continuum

Computation no longer lives in one place. It flows from where data is born to where it can scale.



EDGE

On the device

- Sensors, phones, cars, wearables
- Sub-millisecond latency
- Tiny compute (MCU → mobile NPU)
- Works offline



FOG

Near the device

- Gateways, base stations, on-prem servers
- Single-digit ms latency
- Aggregates many edges
- 5G MEC, AWS Outposts



CLOUD

Far away

- Hyperscale data centers
- 10–200 ms latency
- Effectively unlimited compute
- Long-term storage & training

← lower latency, less data



more compute, more data →

CHALLENGES

- Ultra-Big data
- Infeasible and inefficient to handle with cloud services because of computing and communication recourses.
- Using centralized approached where all the data analysis works are executed on cloud servers.
- Having access from different types of users and devices
- Addressing the proper device, sensor or any components for purposes

Heterogeneity
Openness
Security
Failure Handling
Concurrency
Quality of Service
Scalability
Transparency



ireless link



IOT CHARACTERISTICS

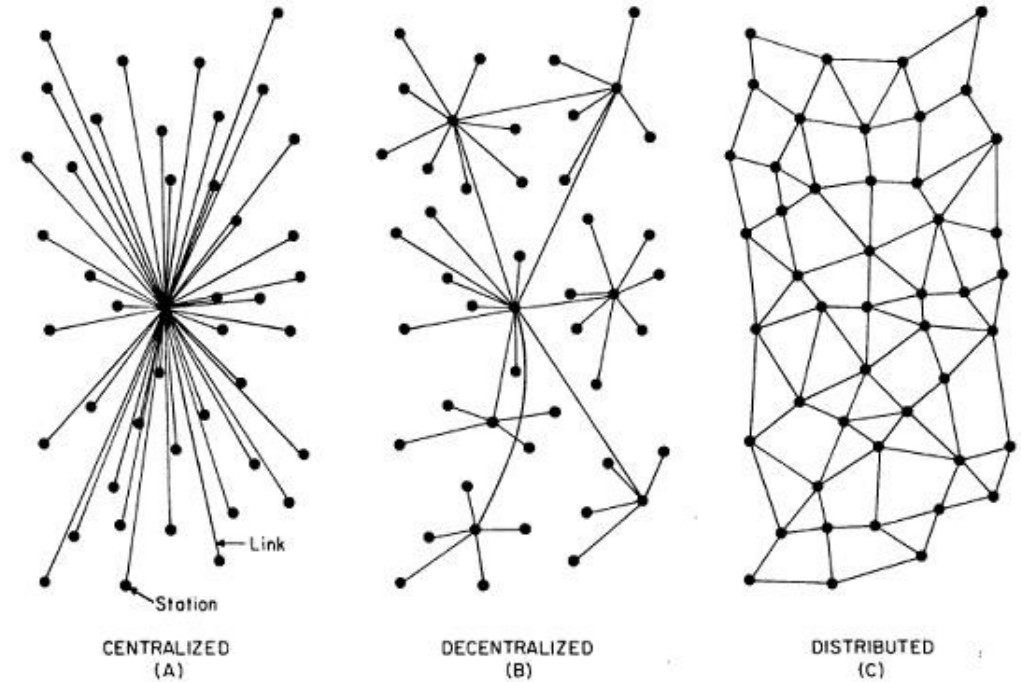
- Intelligence
- Connectivity
- Dynamic Nature
- Enormous scale
- Sensing
- Heterogeneity
- Security
- Zero-Configuration

SOLUTION



IoT MAIN MODELS

- There are mainly 3 types of system architectures utilized by modern software applications which in turn determine the architecture of an application:
 - Centralized
 - Decentralized
 - Distributed



DECENTRALIZED

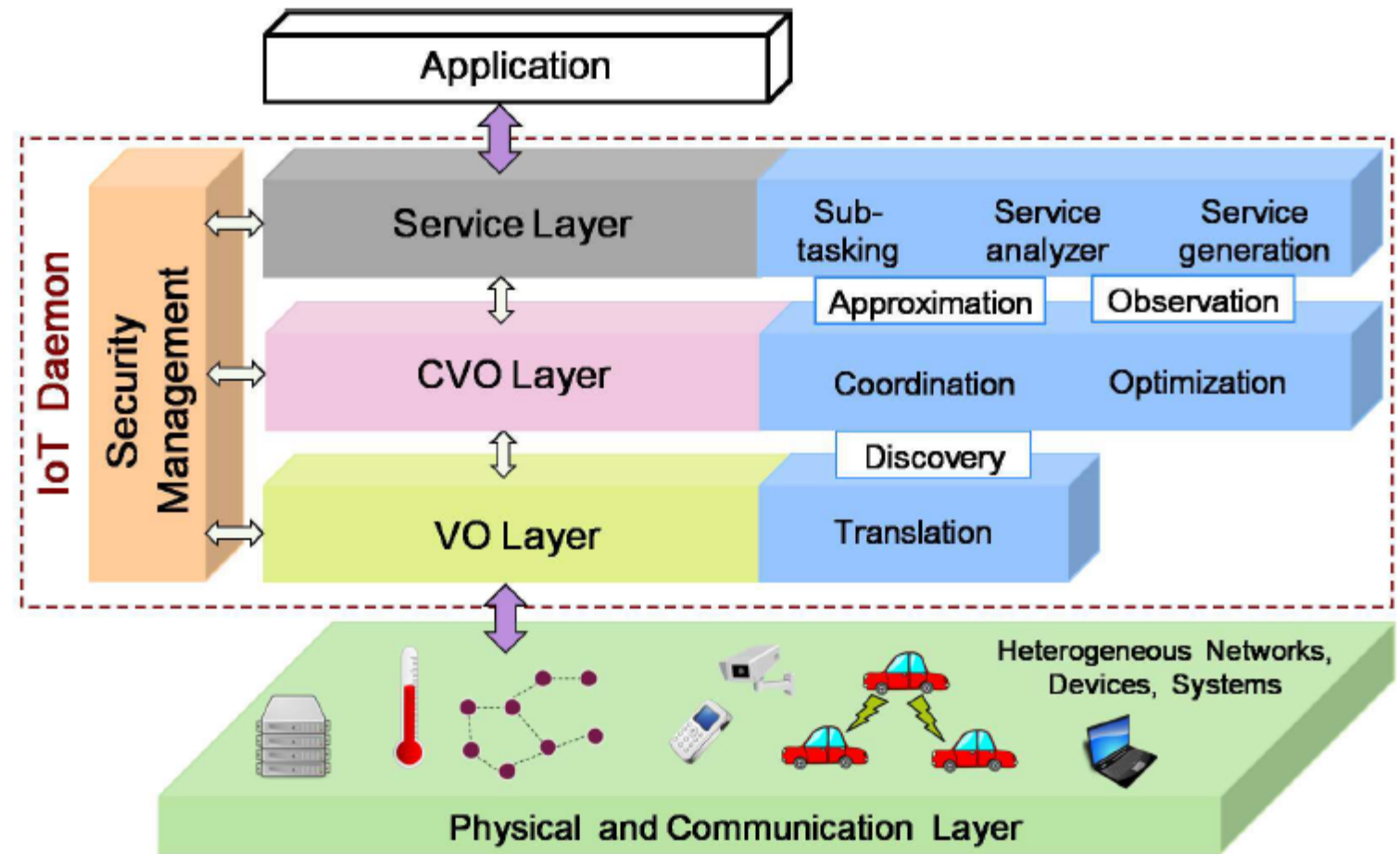
- In a decentralized, non-distributed (or co-located) system, all the parts of the system are in the same physical location.
- This eliminates the biggest problem of single point of failure with centralized network.
- Decentralized systems form the basis of parallel and cluster computing where different processors on the network can execute the same instruction set on different parts of the same dataset (shards) or different dataset.
- Features:
 - **Individual goals:** the nodes are working to achieve individual goals which may need to be coordinated to get the end result (eg map reduce)
 - **Loosely coupled:** the nodes on the network should be loosely coupled
 - **Direct synchronization with other nodes on the network:** the nodes on the network should be synchronized via a common clock
 - **Shared memory:** the nodes on the network should use a shared memory space to store intermediate state and data
 - **Geographical distribution:** the nodes on the network should be collocated, with minimum geographical distribution for data redundancy if required
 - **Homogeneity:** the nodes on the network should ideally be autonomous and heterogeneous in nature

DISTRIBUTED

- In a distributed network the nodes are not collocated but distributed geographically also It avoids the centralization completely.
- A node can either be a communication endpoint or a point of communication redistribution, linking to other nodes.
- Features:
 - **Common goal:** the nodes are working to achieve a common goal that cannot be achieved using a single processor
 - **Loosely coupled:** the nodes on the network should be loosely coupled and should be able to communicate via a messaging system using a standard protocol
 - **No direct synchronization with other nodes on the network:** the nodes on the network should not be synchronized via a common clock
 - **Distributed memory:** the nodes on the network should not have a shared memory space if possible, but shared distributed memory space for the entire system can be provided as an abstraction
 - **Geographical distribution:** the nodes on the network should be geographically distributed and should avoid any centralization
 - **Autonomy and heterogeneity:** the nodes on the network should be autonomous and heterogeneous in nature

DIAT: A SCALABLE DISTRIBUTED ARCHITECTURE FOR IoT

- Virtual Object Layer (VOL), for object virtualization
- Composite Virtual Object Layer (CVOL), for service composition and execution
- Service Layer (SL), for service creation and management



Edge AI & TinyML - Bringing Inference to the Device

Why run ML on the device?

Latency

Sub-10 ms response — no round trip

Privacy

Raw data never leaves the device

Bandwidth

Send insights, not gigabytes of video

Resilience

Keep working when the network drops

Where it runs in 2026



Apple Neural Engine

38 TOPS on iPhone 15 Pro / M-series



Google Tensor / Edge TPU

On-device Gemini Nano, Coral devkits



NVIDIA Jetson

Robotics, drones, autonomous machines

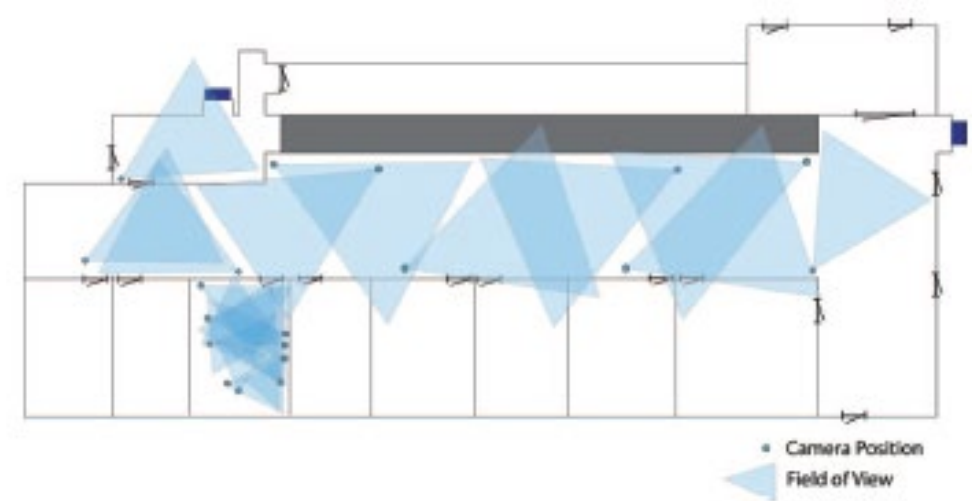


TF Lite • ONNX Runtime • MLX

Quantized models down to ~kilobytes

EXAMPLE: DISTRIBUTED COMPUTING IN VIDEO SENSOR NETWORK

- Video Surveillance:
- In this case target is to store video sequence when critical event occurs. And the cameras have motion detection feature
- Approaches:
 - Centralized approach: All data are streamed back to the server.
 - Distributed approach: Intra processing stage and Inter Processing stage

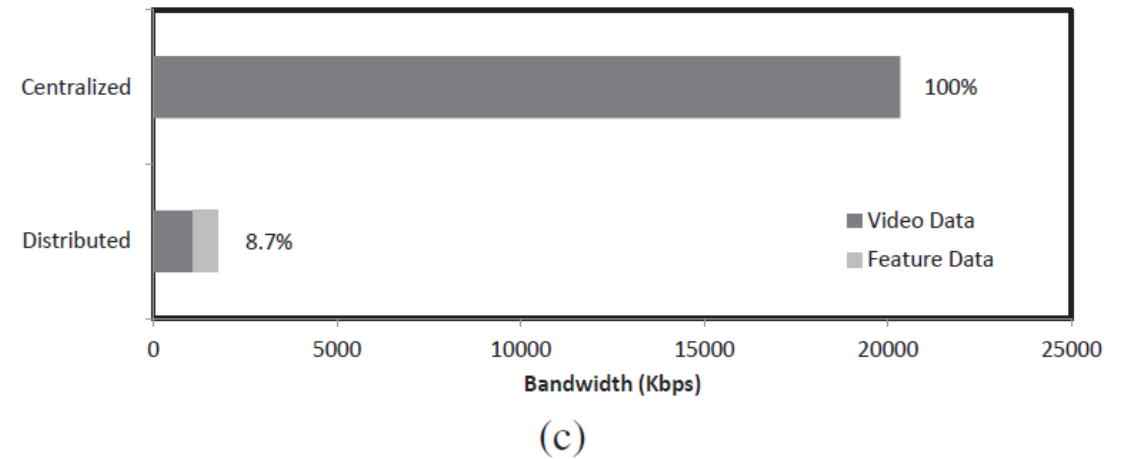


(a)



EXAMPLE: DISTRIBUTED COMPUTING IN VIDEO SENSOR NETWORK

- As showing in figure, with Distributed Approach, 91.3% of the transmission bandwidth can be saved.





BIGDATA

Prof. Tim Wood & Prof. Roozbeh Hagnazar

WHAT'S BIG DATA?

No single definition; here is from Wikipedia:

- **Big data** is the term for a collection of data sets so large and complex that it becomes difficult to process using on-hand database management tools or traditional data processing applications.
- The challenges include capture, curation, storage, search, sharing, transfer, analysis, and visualization.
- The trend to larger data sets is due to the additional information derivable from analysis of a single large set of related data, as compared to separate smaller sets with the same total amount of data, allowing correlations to be found to "spot business trends, determine quality of research, prevent diseases, link legal citations, combat crime, and determine real-time roadway traffic conditions."

BIG DATA: 3V's

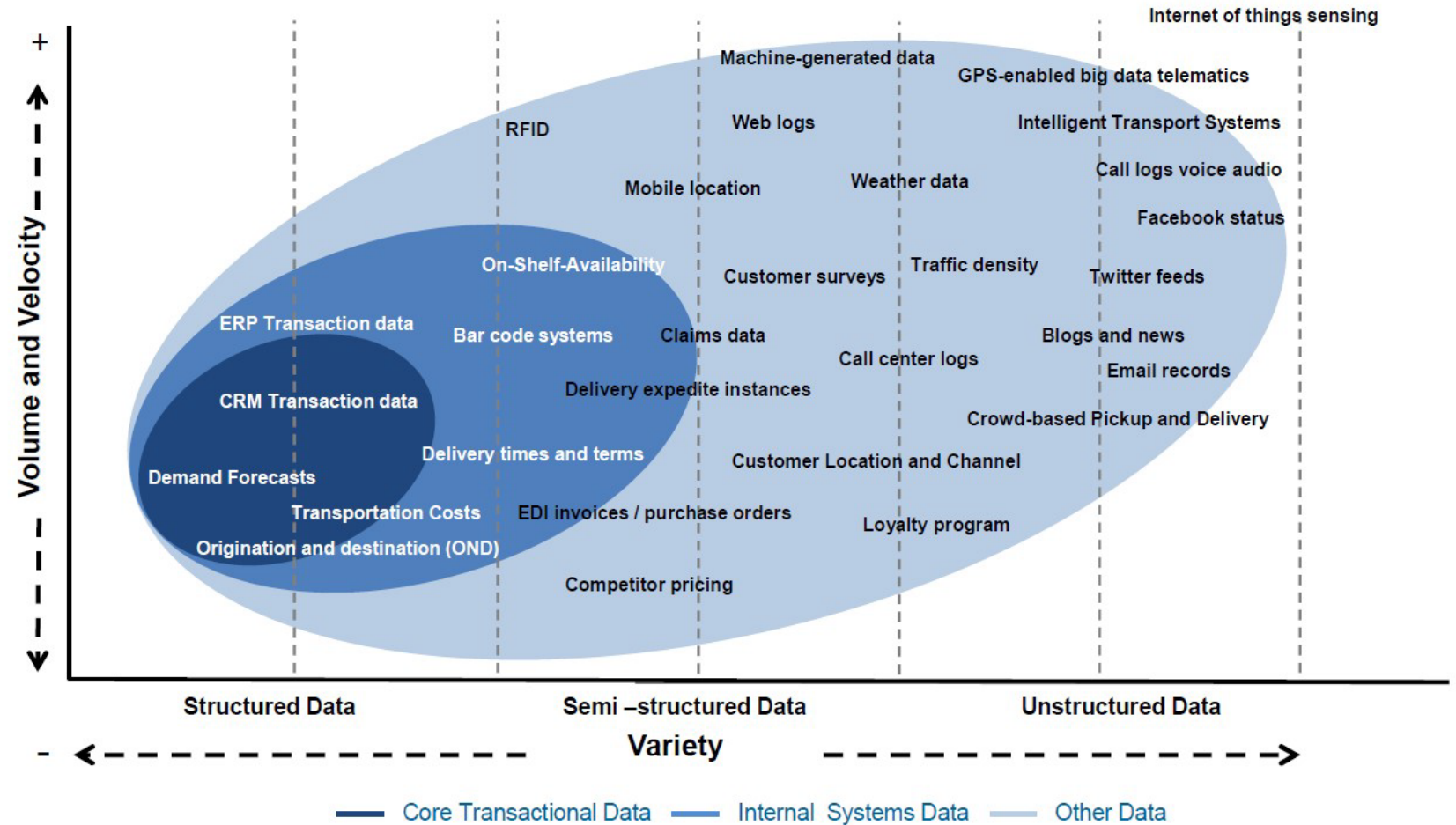
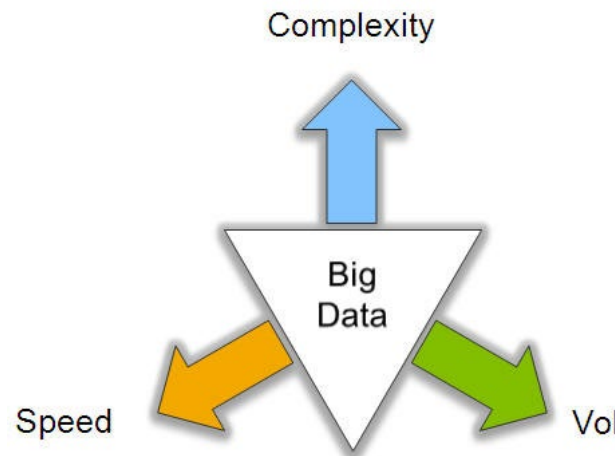
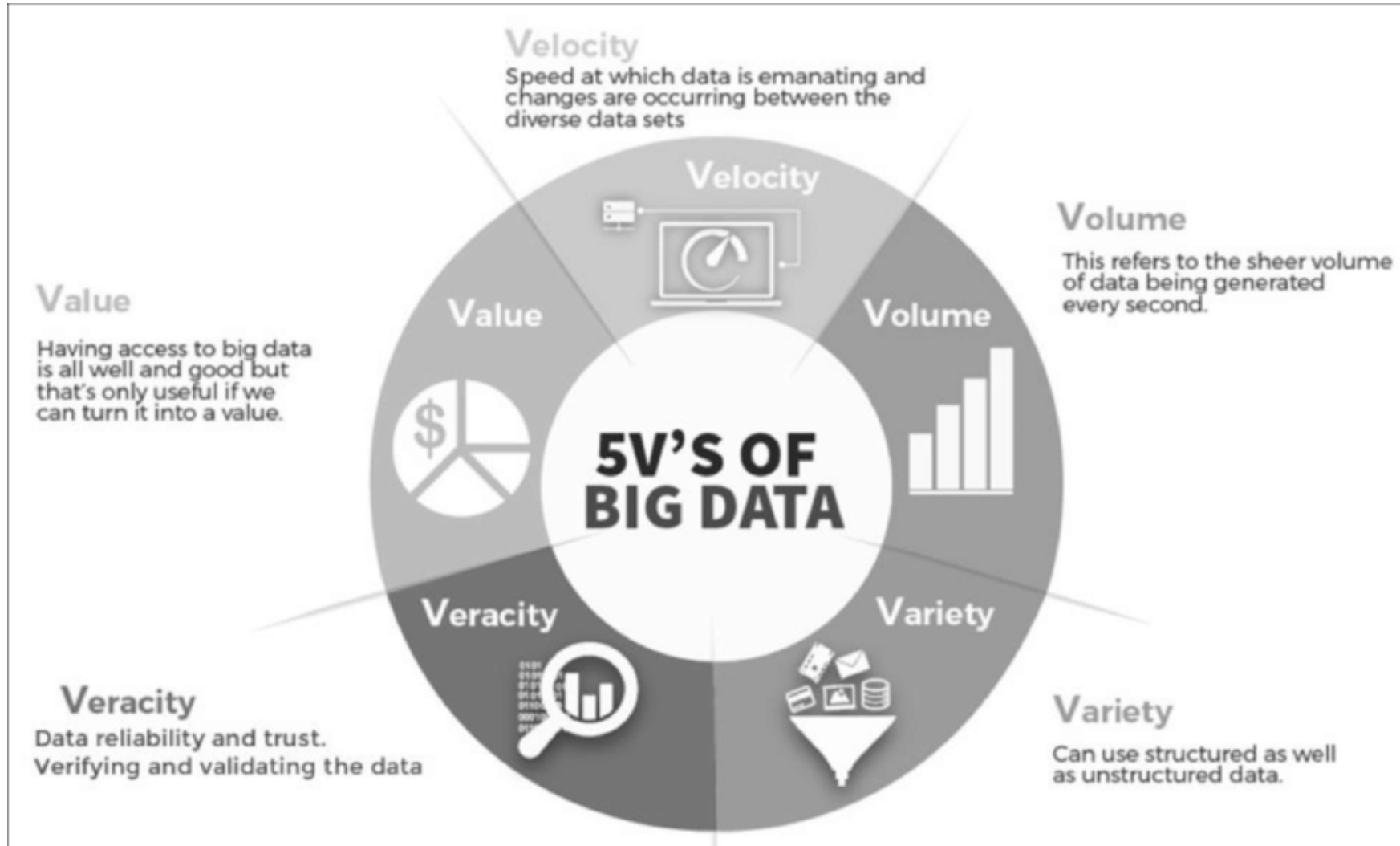


Figure 1. SCM Data Volume and Velocity vs. Variety

BIG DATA 5Vs



The Scale of Distributed Data

These numbers are why distributed systems exist.



5.5B+

internet users

≈ 68% of humanity online



1B+

ChatGPT weekly users

and growing — plus Gemini, Claude, etc.



180+

ZB created in 2025

1 ZB = 1 trillion GB



18B+

connected IoT devices

more devices than humans, by ~2×



10^{25}

FLOPs to train GPT-4

weeks of compute on tens of thousands of GPUs



100k+

tokens/sec served

single LLM endpoint at peak (vLLM, TRT-LLM)

CLOUD COMPUTING AND BIG DATA

- One of the vital issues that organizations face with the **storage and management** of Big Data is the **huge amount of investment** to get the required hardware setup and software packages.
- Some of these resources may be **overutilized or underutilised with varying requirements overtime**. We can overcome these challenges by providing a set of computing resources that can be shared through cloud computing.
- Cloud platforms provide a **shared infrastructure** and **pay-as-you-go model** that lowers cost. This has helped enable the “big data revolution”

IN-MEMORY TECHNOLOGY FOR BIG DATA

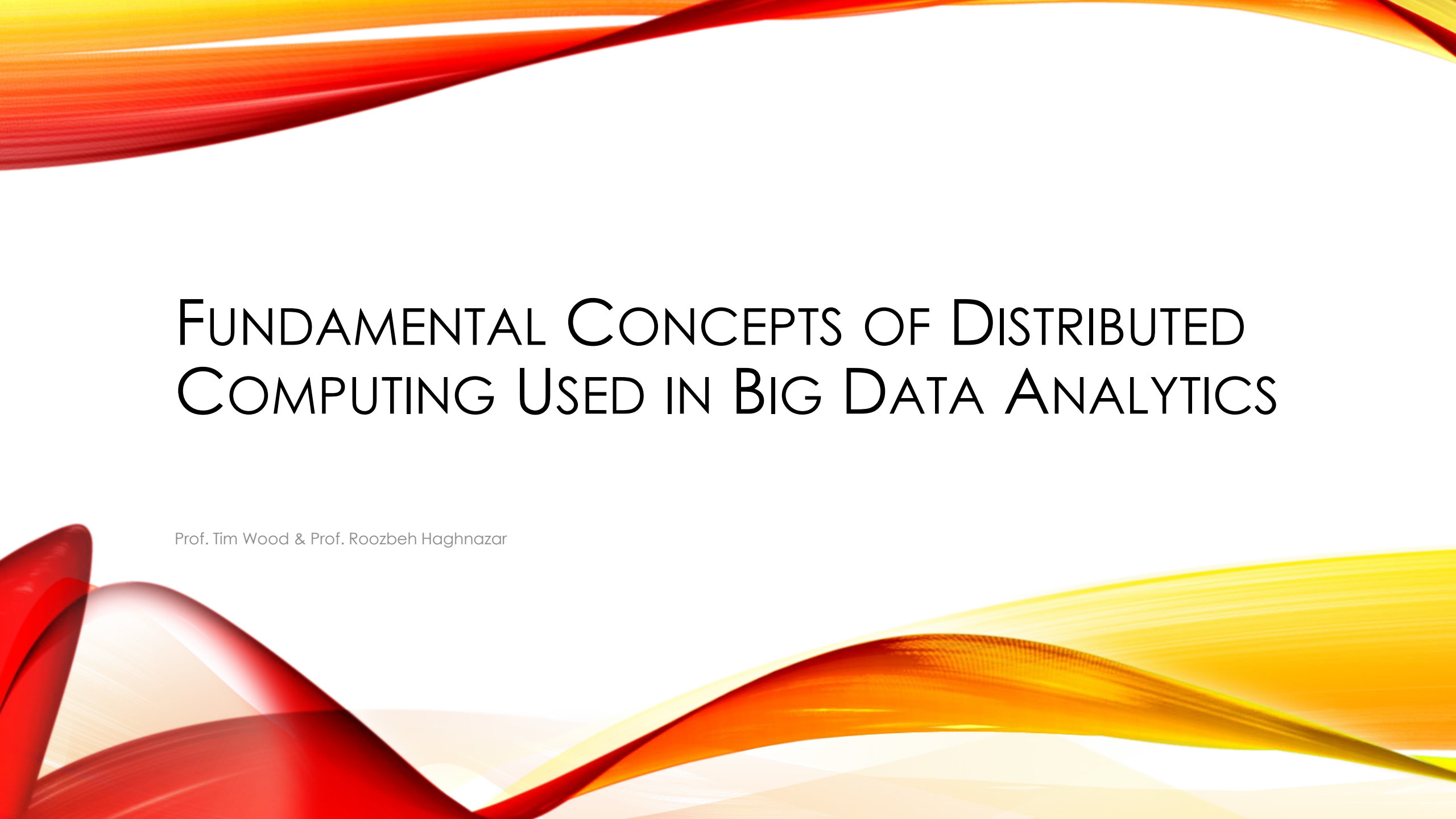
- Disks are really really slow! This might prevent us from analysing data in **real-time**
- **In-memory** big data computing tools overcome this issue by keeping the important data in memory.
 - But this isn't always possible! **Why not?**

What types of applications or analytics are a good fit for in-memory processing?



BIG DATA TECHNIQUES

- To analyze the datasets, there are many techniques available, some of which are as follows:
 - Massive Parallelism
 - Data Distribution
 - High-Performance Computing
 - Task and Thread Management
 - Data Mining and Analytics
 - Data Retrieval
 - Machine Learning
 - Data Visualisation



FUNDAMENTAL CONCEPTS OF DISTRIBUTED COMPUTING USED IN BIG DATA ANALYTICS

Prof. Tim Wood & Prof. Roozbeh Haghazari

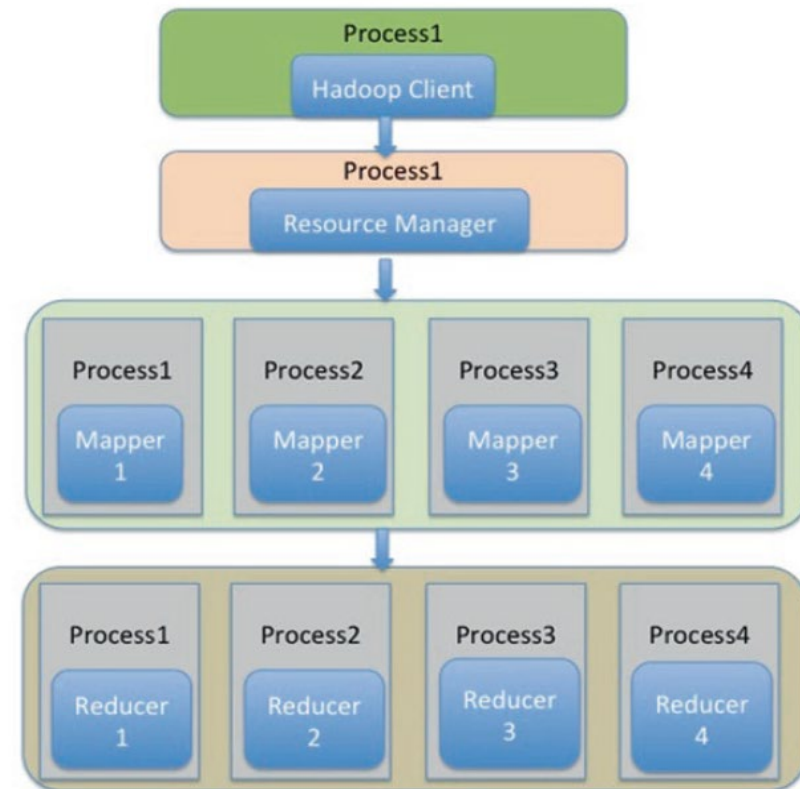


BIG DATA AND...

- **Heterogeneity**
- **Openness**
- **Security**
- **Failure Handling**
- **Concurrency**
- **Quality of Service**
- **Scalability**
- **Transparency**

MAP REDUCE / HADOOP

- The first popular “big data” platform for the cloud
- Can install **Apache Hadoop** on your own **IaaS** VMs/containers
- Or use a **PaaS** version:
 - AWS Elastic Map Reduce, Google Dataproc, Azure HDInsight
- MapReduce uses distributed systems concepts to achieve high scalability
 - Partitioning, failure detection, data replication for performance and reliability...
- MapReduce relies on a cloud infrastructure to provide its computational and storage resources



Multiprocessing model in the Hadoop runtime environment

Beyond Hadoop - The Modern Data Stack

LEGACY (2006–2015)

Hadoop / MapReduce / HDFS

- Disk-bound batch jobs (minutes → hours)
- Rigid map → shuffle → reduce shape
- Java-heavy, painful to iterate
- Storage and compute tightly coupled
- Effectively in maintenance mode

MODERN (2020 →)

Lakehouse + Unified Compute



Apache Spark

In-memory batch + structured streaming



Apache Flink

True low-latency stateful streaming



Iceberg / Delta / Hudi

Open table formats — ACID on object storage



Databricks • Snowflake

Decoupled storage and elastic compute



Ray

Unified train + tune + serve (used by OpenAI)

MAP REDUCE **QUALITY OF SERVICE**

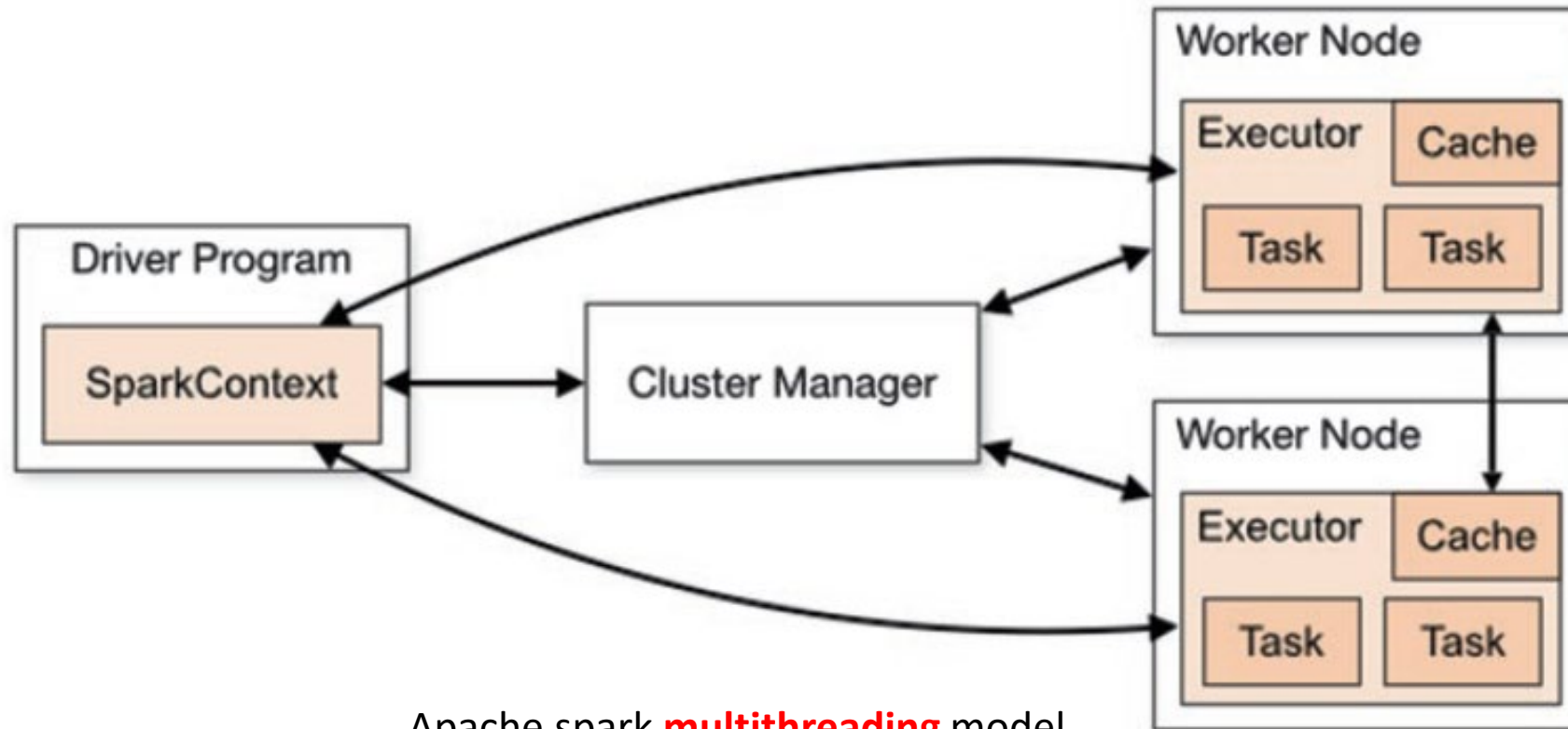
- We already discussed how Hadoop/MR can have different scheduling algorithms to decide which tasks to process when there are several jobs
- Straggler issue: It can be difficult to tell the difference between a really slow and a failed replica
 - MapReduce solution:

STREAM PROCESSING

- MR/Hadoop are for **batch** processing
 - Long running jobs (minutes, hours total)
- Sometimes you want **stream** processing
 - Continuously arriving data with millisecond scale response
- **Storm** and **Spark** are basically Hadoop for streams
 - Define a graph of processing nodes
 - Stream data through the graph
 - Manage the workers (each executing a part of the graph)
 - Detect failure, carefully buffer data in queues, etc

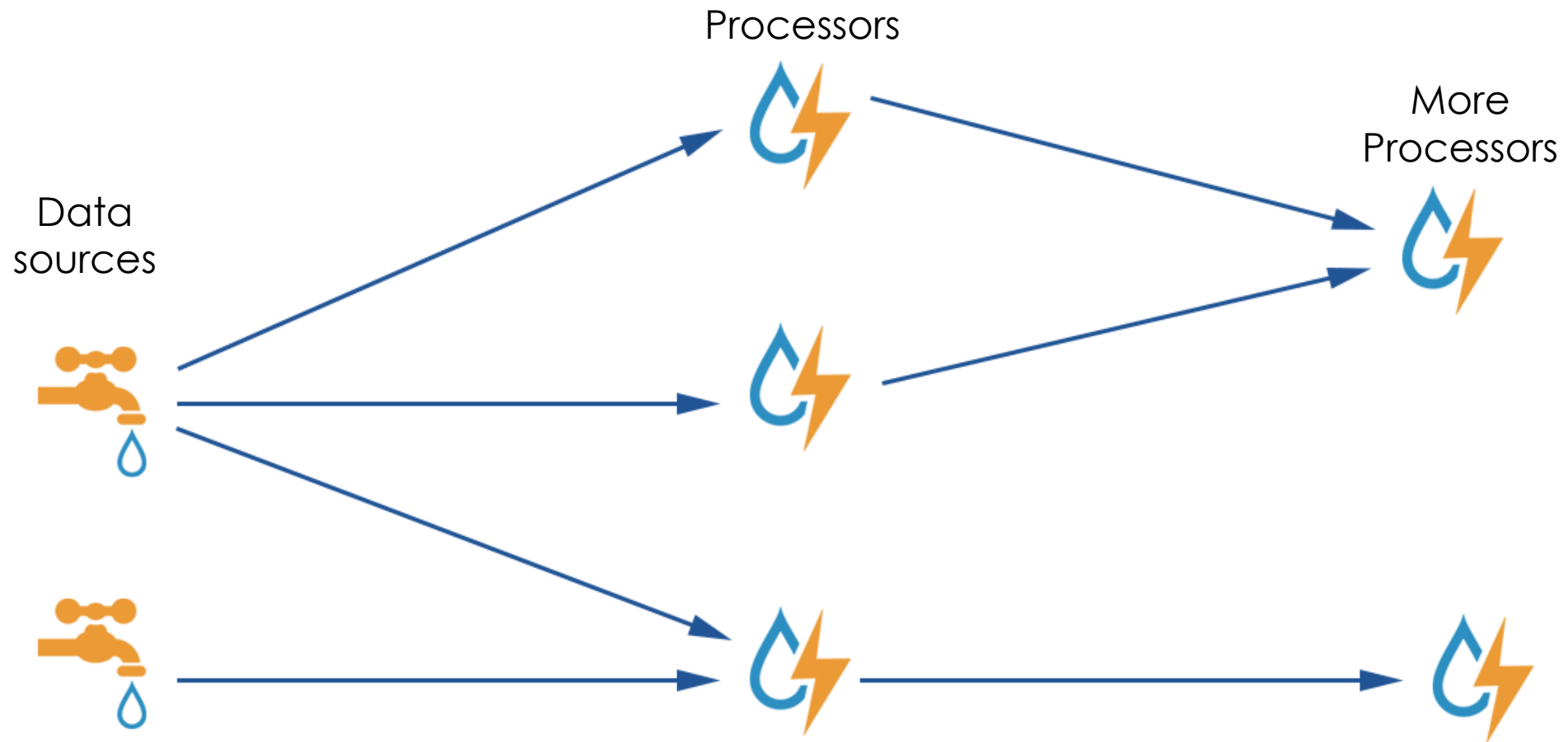


SPARK REQUEST HANDLING MODLE



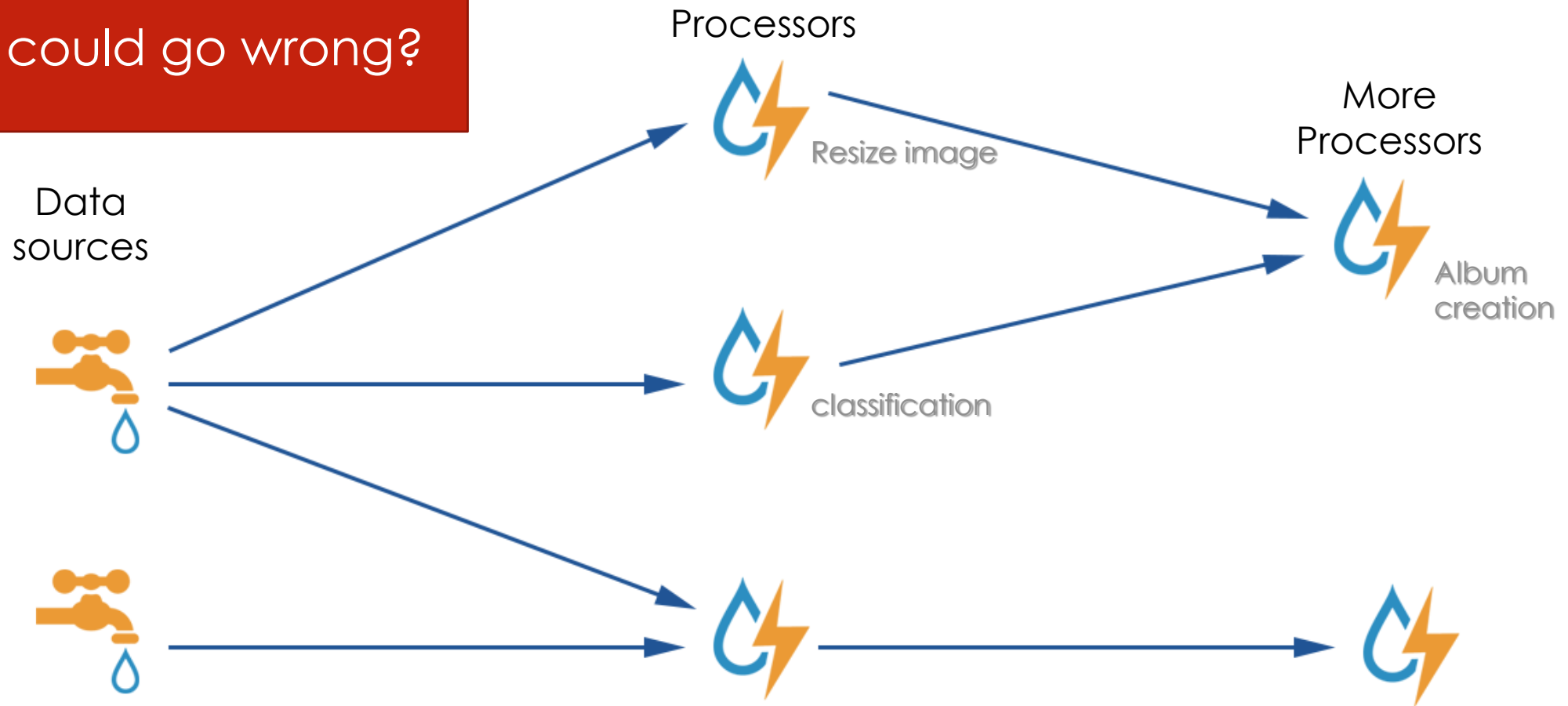
Apache spark **multithreading** model

STREAM PROCESSING



STREAM PROCESSING

What could go wrong?

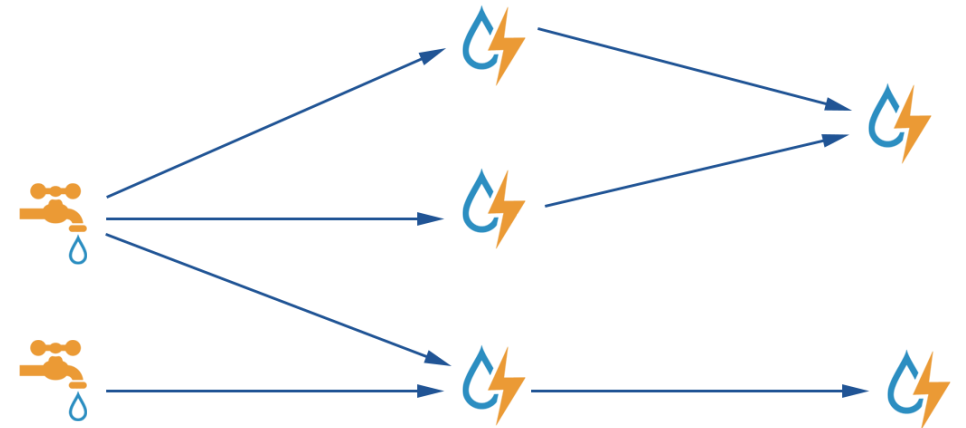


STREAM PROCESSING

Problem	Why it's harder than batch
A processor crashes mid-stream	In Hadoop you restart the job. In streaming, data is still arriving while you're recovering - what happens to it?
Slow processor (backpressure)	If "classification" is slower than "resize", a queue builds up forever. Memory explodes.
Duplicate processing	If a node retries after a failure, the photo might get classified twice → appears in two albums
Out-of-order arrival	Photo B arrives before Photo A because of network jitter - does the album order break?
Partial failure	Resize succeeded, classification crashed. Do we resize again? Skip? Retry forever?
Exactly-once is hard	"Album creation" should fire once per photo. But if any node retries, we might create the album twice.

STORM **FAULT TOLERANCE**

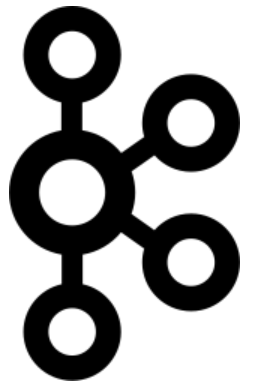
- Data is constantly arriving -> handling faults is more difficult
- We want to understand the **fault tolerance semantics** provided by the stream processing system
- **Best effort**: no guarantees
- **At least once**: a data element will be fully processed, but it might be processed several times
- **Exactly once**: a data element will be fully processed, precisely one time by each step



State
Replication

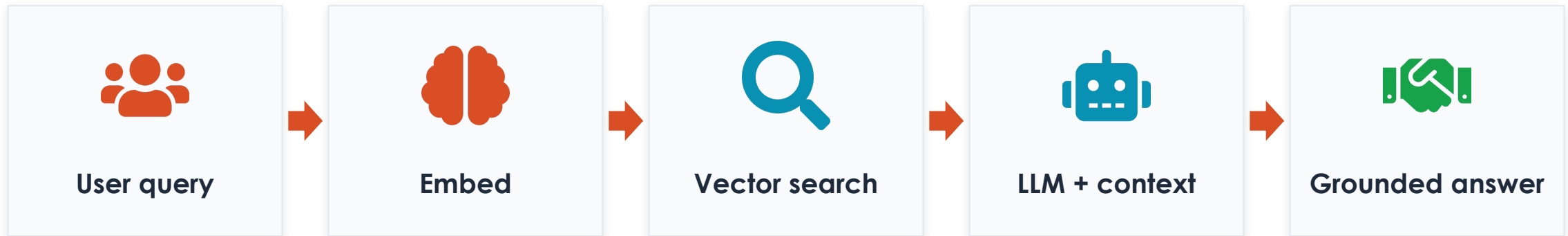
STREAM PROCESSING **SCALABILITY**

- Need to manage scalability of stream processing workers
- How to make data available to workers?
 - Hadoop Distributed File System would be too slow
- **Kafka** Message Queue
 - Distributed queue of requests
 - Queue can be scaled up and down as needed
 - Can support large numbers of workers accessing data concurrently
 - Currently uses **Zookeeper** for consistency, next version will use **Raft**



Kafka

Vector Databases & Retrieval Augmented Generation (RAG)



Why it matters for distributed systems

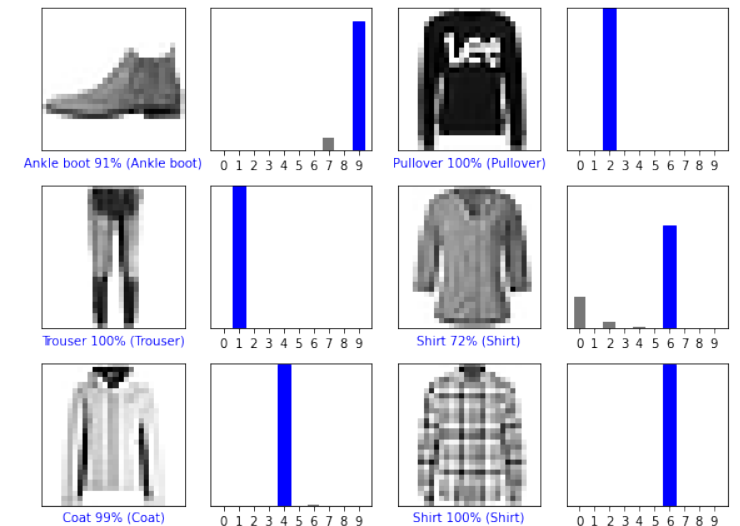
- Billions of high-dim vectors → sharding & replication
- Approximate nearest-neighbor (HNSW, IVF) at scale
- Latency-critical: embeddings → search → LLM in <1s
- Hot read path, cold write path — classic distsys pattern

Vector stores you'll see in industry

Pinecone	Managed, hosted
Weaviate	Open-source, GraphQL-native
Milvus	OSS, GPU-accelerated
pgvector	Postgres extension — “just enough”
Qdrant	Rust-based, fast filters

TENSOR FLOW

- Library from Google primarily for training and running Deep Learning models
 - Designed in part by Jeff Dean (watch his talk!)
- Training machine learning models takes a lot of data
- Doing training on a single machine can be too slow
 - Or a single machine may not have enough memory
- TensorFlow handles all the ML math and helps with distributing the training and inference tasks across multiple devices

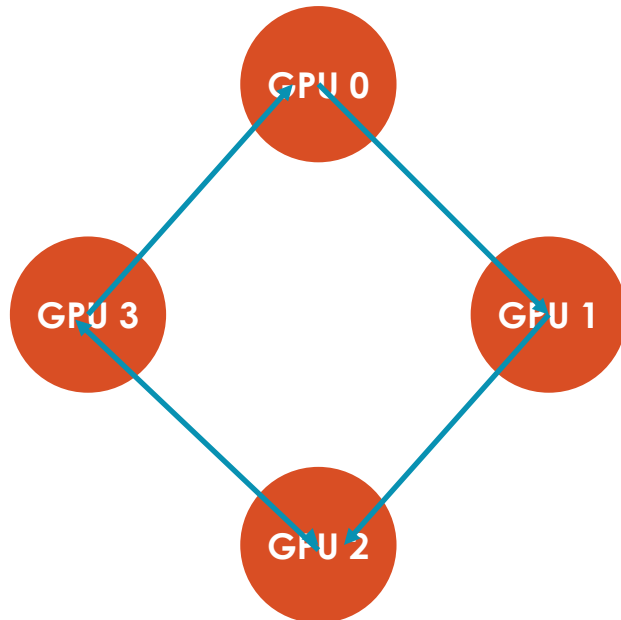


GPU Clusters & Collective Communication

Distributed ML is a coordination problem. Sound familiar?

Ring All-Reduce

Every GPU averages gradients with every other GPU — without a central server.



The stack that makes it fast

NVLink / NVSwitch

900 GB/s GPU-to-GPU inside one node

InfiniBand / RoCE

400–800 Gb/s between nodes

NCCL

NVIDIA collective comm. library — all-reduce, all-gather, broadcast

PyTorch DDP / FSDP

Wraps NCCL — what you actually write code against

Megatron-LM, DeepSpeed

3D parallelism for trillion-parameter models

Training Frontier LLMs - 3D Parallelism

One copy of GPT-4 doesn't fit on a single GPU. Not even close.

GPT-4 has roughly **1.7 trillion** parameters. At FP16 (16-bit numerical format), that's about 3.4 terabytes of weights. An H100 has 80 GB of memory. So how do you even *fit* the model, let alone train it? You split it three ways at once.



Data Parallel

Same model copy on each GPU. Different mini-batches. Sync gradients via all-reduce.



Tensor Parallel

Split a single matrix multiply across GPUs. Heavy intra-layer communication.



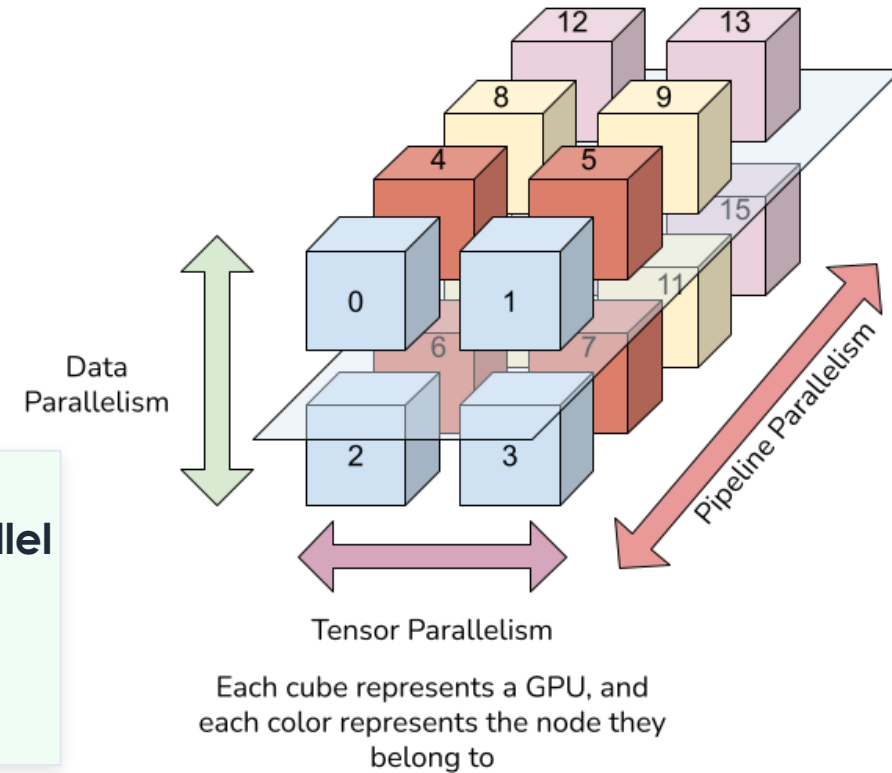
Pipeline Parallel

Different layers on different GPUs. Micro-batches flow through the pipeline.



Real numbers from frontier models

Llama 3 (405B): ~16,000 H100s · ~30M GPU-hours. DeepSeek-V3: 671B params, MoE, ~\$5.5M training cost reported.



LLM Inference at Scale - Harder Than It Looks

Training is a one-time cost. Inference is forever.

Why inference is hard

Autoregressive

Generate one token at a time
can't fully parallelize a single request

KV cache grows

Memory per request scales with output length

Bursty traffic

10× spikes during peak hours, idle GPUs cost real money

Tail latency

P99 matters more than mean for chat UX

Heterogeneous

Short prompts and long generations share the same GPU

How modern systems solve it

Continuous batching

Pack new requests into running batches every step

PagedAttention (vLLM)

OS-style paging for the KV cache
no fragmentation

Speculative decoding

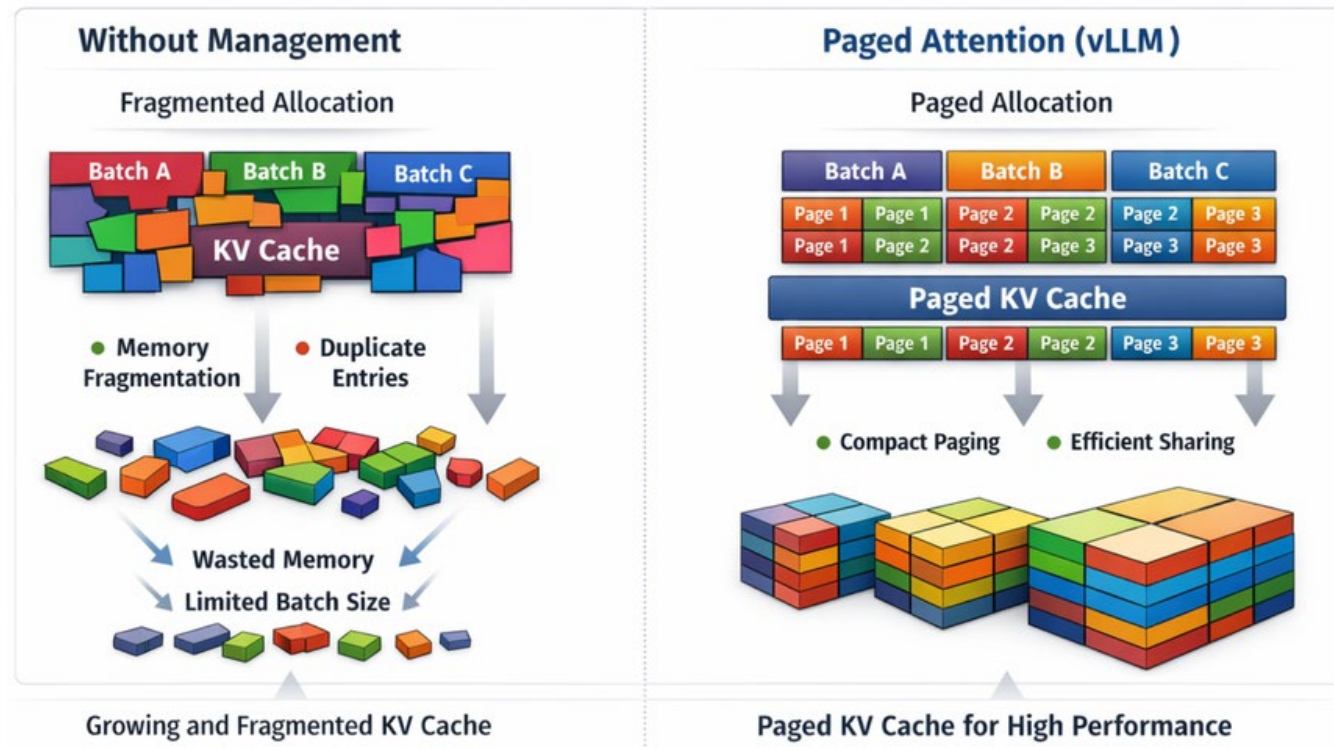
Small draft model proposes tokens, big model verifies

Quantization

FP8 / INT4 - smaller, faster, ~same quality

TensorRT-LLM, SGLang

Production stacks running today's biggest endpoints



the dirty secret of LLM serving is that **memory, not compute, is usually the bottleneck.**

TENSOR FLOW **HETEROGENEITY**

- Many ML tasks are highly parallelizable 😊
 - But they require a lot of data -> high communication costs ☹
- GPUs are very good for this
- A network of distributed GPUs is even better...
- But custom hardware is best!

- TPU = Tensor Processing Unit
 - Customized processor specifically for tensor flow

- TensorFlow software library needs to support all of these, possibly spread across multiple servers

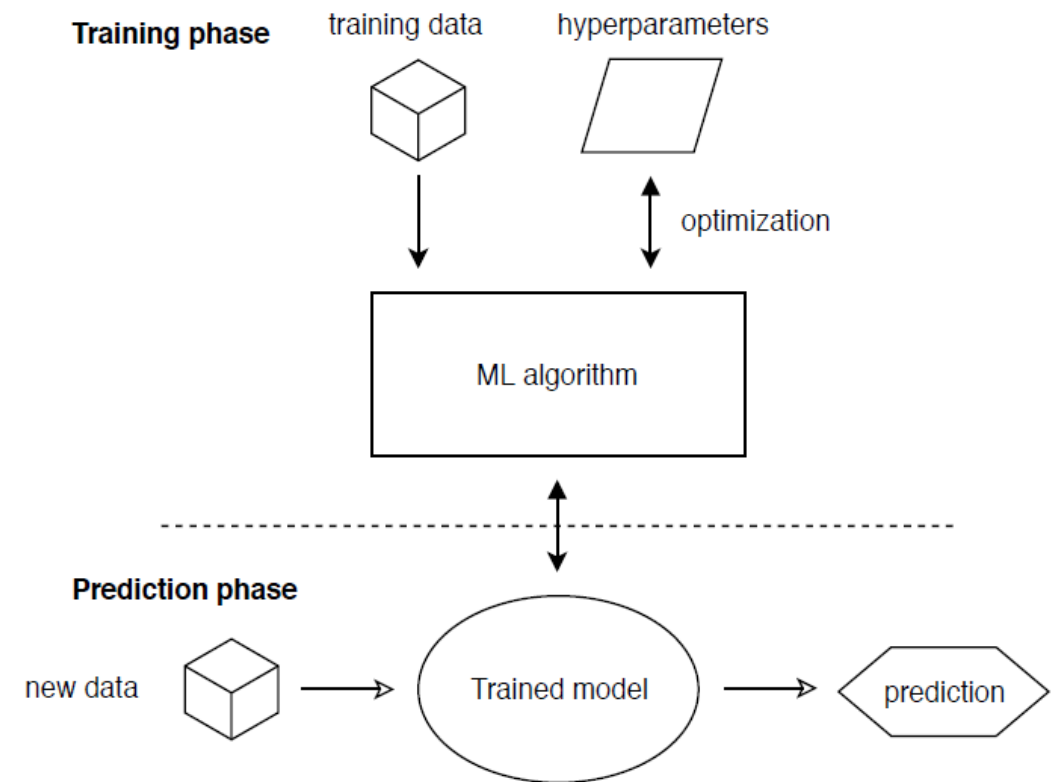


MACHINE LEARNING AND DISTRIBUTED SYSTEMS

Prof. Tim Wood & Prof. Roozbeh Haghazari

GENERAL ML PROBLEM

- General Overview of Machine Learning. During the training phase a ML model is optimized using training data and by tuning hyperparameters. Then the trained model is deployed to provide predictions for new data fed into the system.
- Aside from choosing a suitable algorithm for a given problem, we also need to find an optimal set of hyperparameters for the chosen algorithm



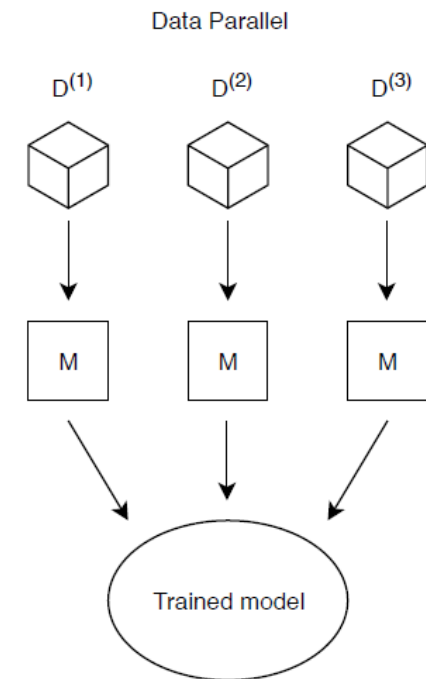


DISTRIBUTION OF TRAINING PHASE

- **Data-Parallel**
- **Model-Parallel**

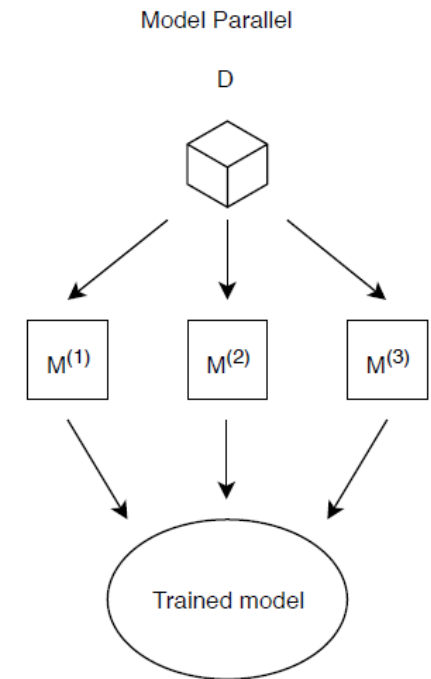
DISTRIBUTION OF TRAINING PHASE

- In the Data-Parallel approach, the data is partitioned as many times as there are worker nodes in the system and all worker nodes subsequently apply the same algorithm to different data sets.
- The same model is available to all worker nodes (either through centralization, or through replication) so that a single coherent output emerges naturally.
- The technique can be used with every ML algorithm with an independent and identically distribution (i.i.d.) assumption over the data samples



DISTRIBUTION OF TRAINING PHASE

- In the Model-Parallel approach, exact copies of the entire data sets are processed by the worker nodes which operate on different parts of the model.
- The model is therefore the aggregate of all model parts.
- The model-parallel approach cannot automatically be applied to every machine learning algorithms because the model parameters generally cannot be split up.



MACHINE LEARNING ALGORITHMS

- We categorize current ML algorithms based on the following three characteristics:
 - **Feedback**, the type of feedback that is given to the algorithm while learning
 - *Supervised learning*
 - *Unsupervised learning*
 - *Semi-supervised learning*
 - *Reinforcement learning*
 - **Purpose**, the desired end result of the algorithm
 - *Anomaly detection*
 - *Classification*
 - *Clustering*
 - *Dimensionality reduction*
 - *Representation learning*
 - *Regression*
 - **Method**, the nature of model evolution that occurs when given feedback
 - *Evolutionary algorithms (EAs)*
 - *Stochastic gradient descent (SGD) based algorithms*
 - Support vector machines (SVMs)
 - Artificial neural networks (ANNs)
 - Perceptrons
 - *Rule-based machine learning (RBML) algorithms*
 - *Topic Models (TM)*
 - *Matrix Factorization*

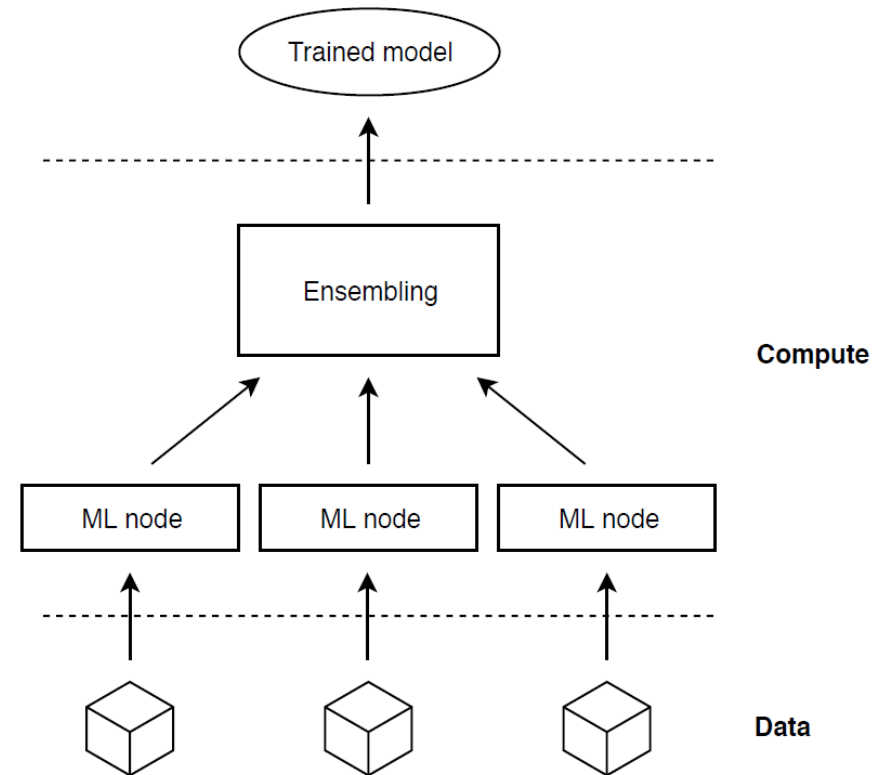


TOPOLOGIES

- Centralized (Ensembling)
- Decentralized (Tree)
- Decentralized (Parameter Server)
- Fully Distributed (Peer to Peer)

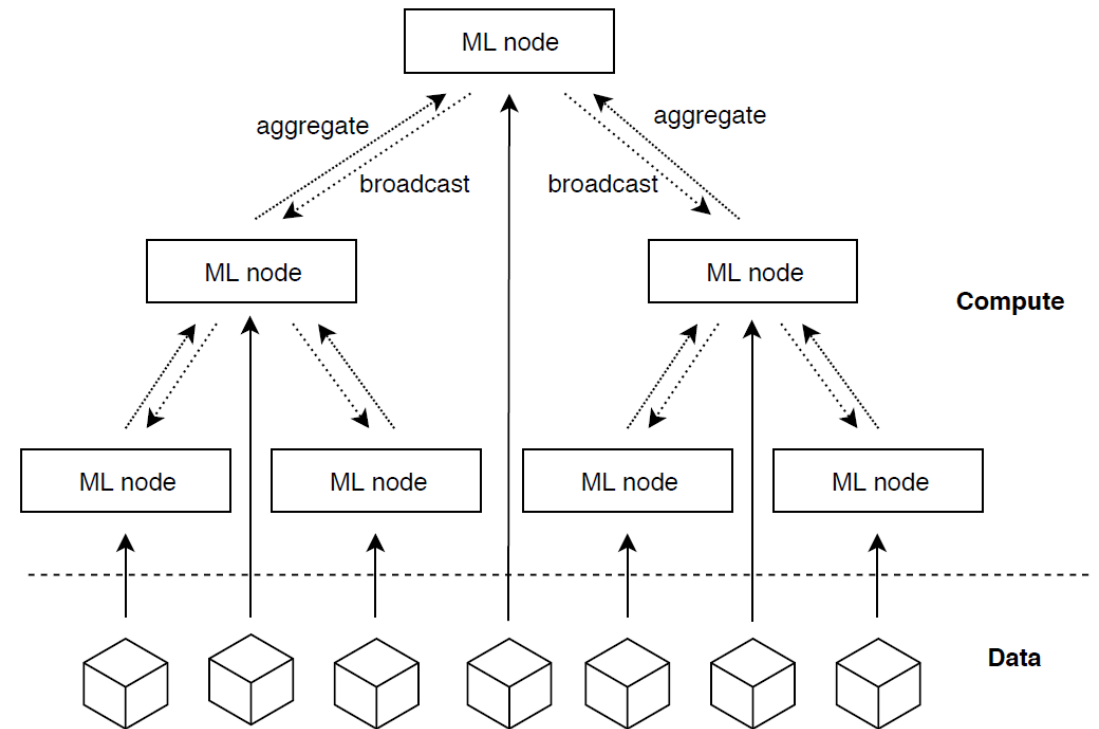
CENTRALIZED (ENSEMBLING)

- employ a strictly hierarchical approach to aggregation, which happens in a single central location.



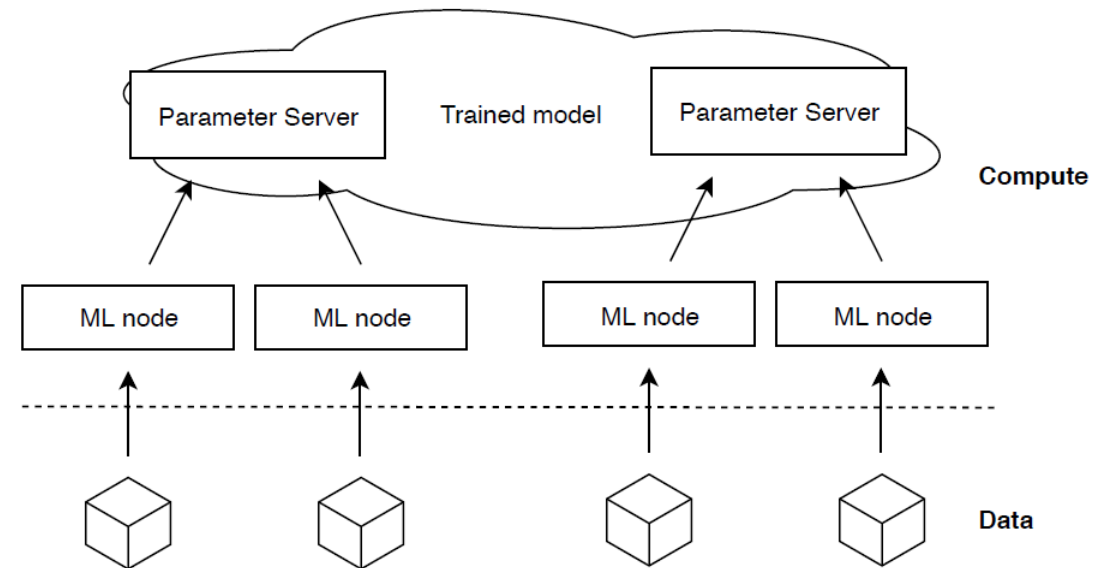
DECENTRALIZED (TREE)

- allow for intermediate aggregation, with a replicated model that is consistently updated when the aggregate is broadcast to all nodes such as in tree topologies



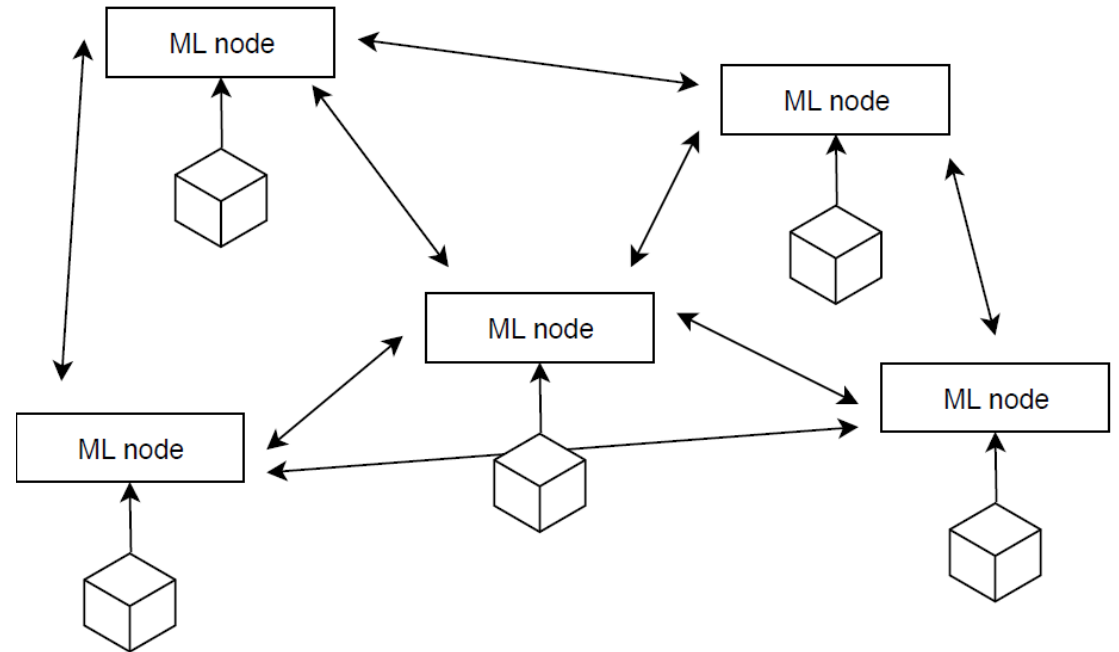
DECENTRALIZED (PARAMETER SERVER)

- allow for intermediate aggregation, with a partitioned model that is sharded over multiple parameter servers



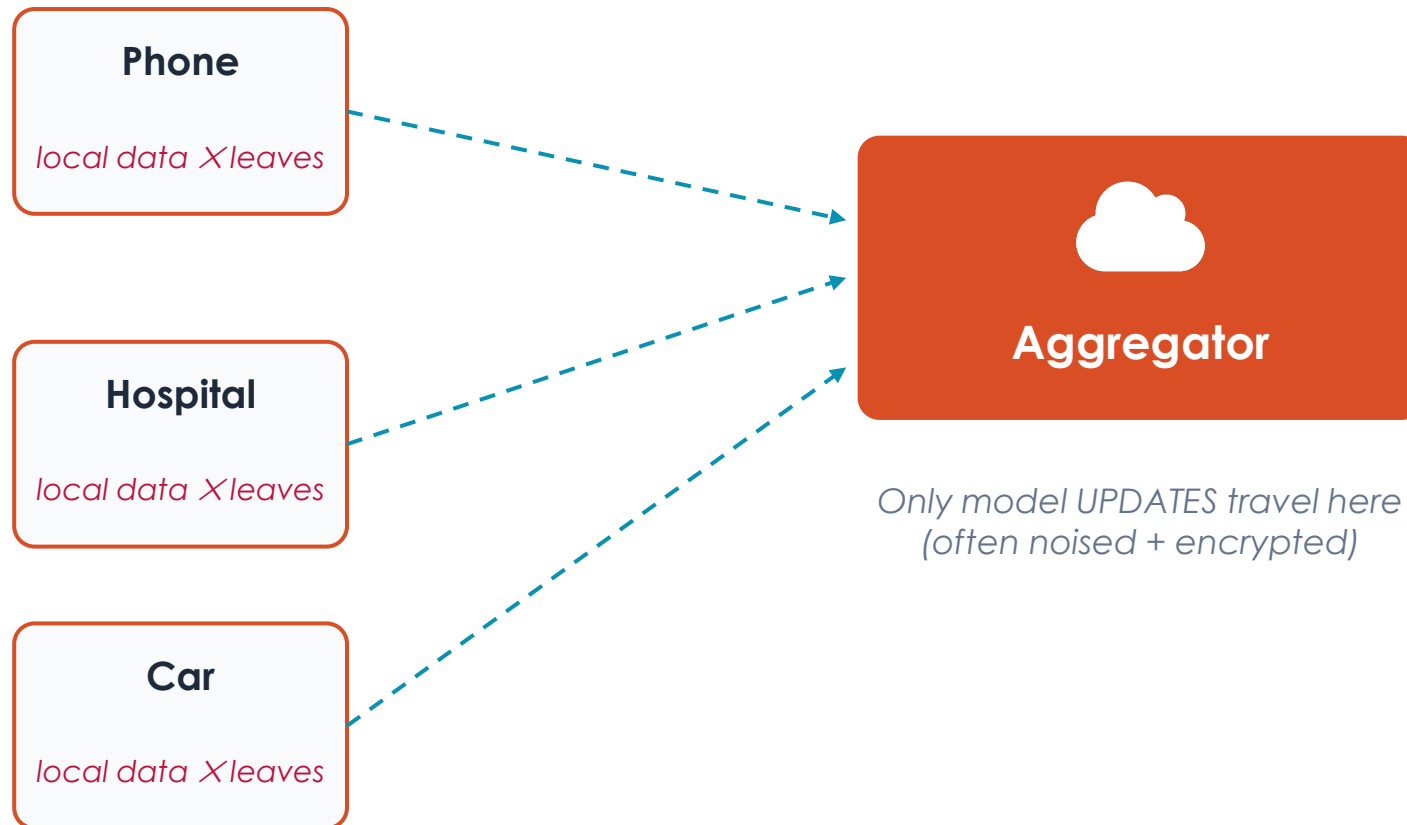
FULLY DISTRIBUTED (PEER TO PEER)

- consists of a network of independent nodes that ensemble the solution together and where no specific roles are assigned to certain nodes.



Federated Learning - Train Without Centralizing Data

Bring the model to the data, not the data to the model.



In production today

Google Gboard

Next-word prediction

Apple

Siri, QuickType, photo classification

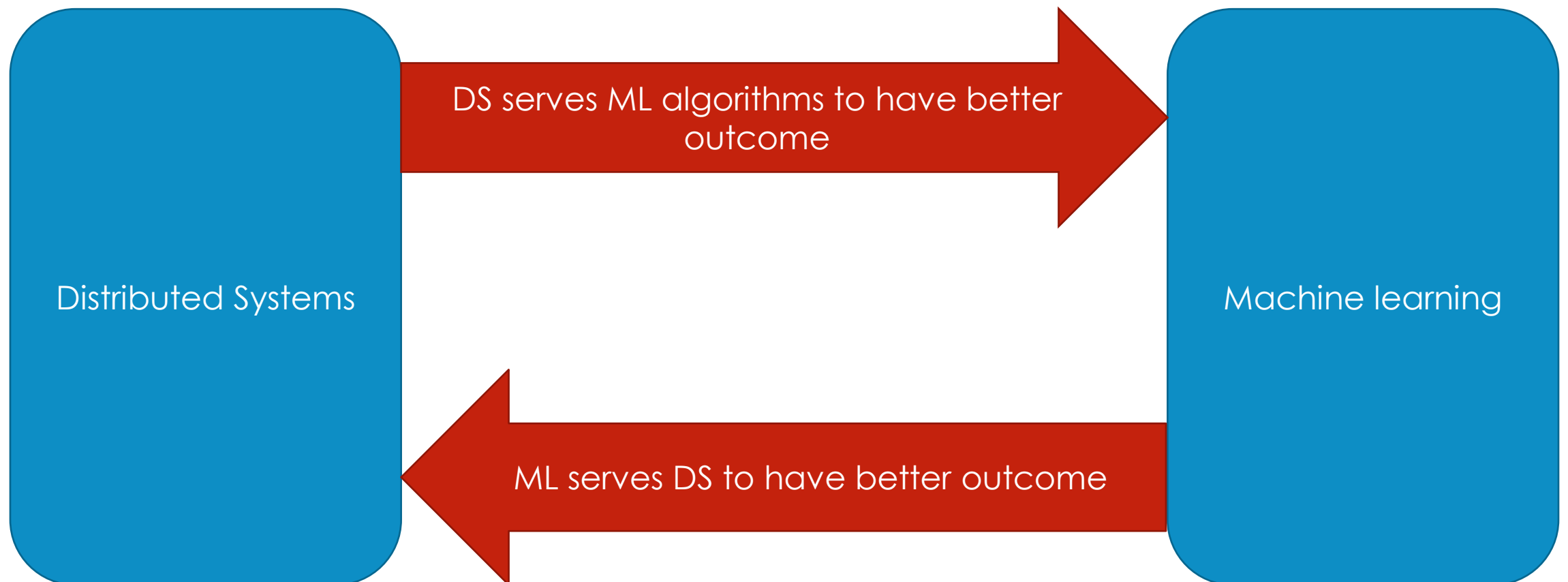
NVIDIA FLARE

Hospital imaging research

Owkin

Cross-hospital cancer models

COEXISTENCE OF DISTRIBUTED SYSTEM AND MACHINE LEARNING



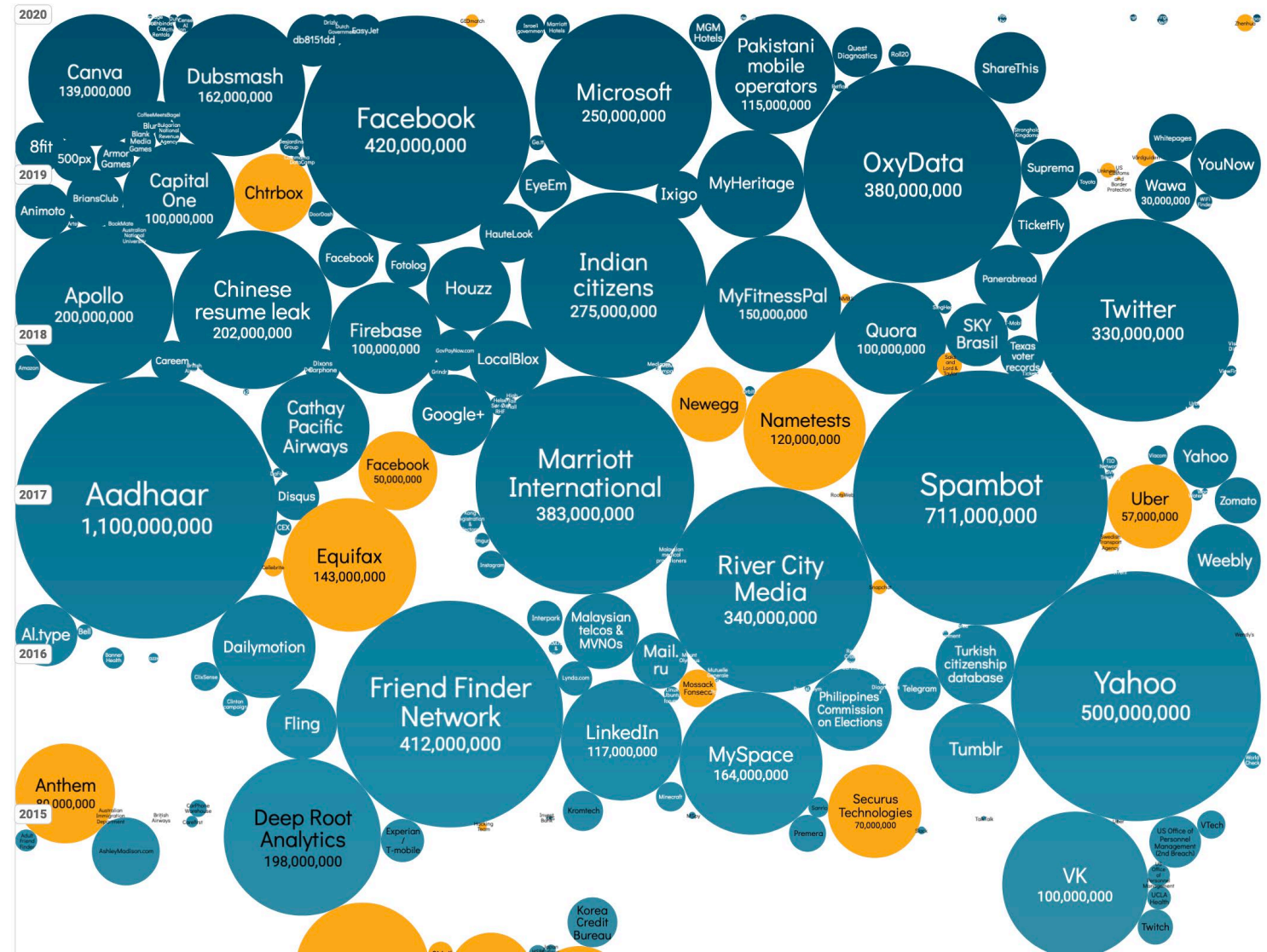


BIG DATA AND CLOUD SECURITY

Prof. Tim Wood & Prof. Roozbeh Haghazari

UPDATED: Dec 2020

search...



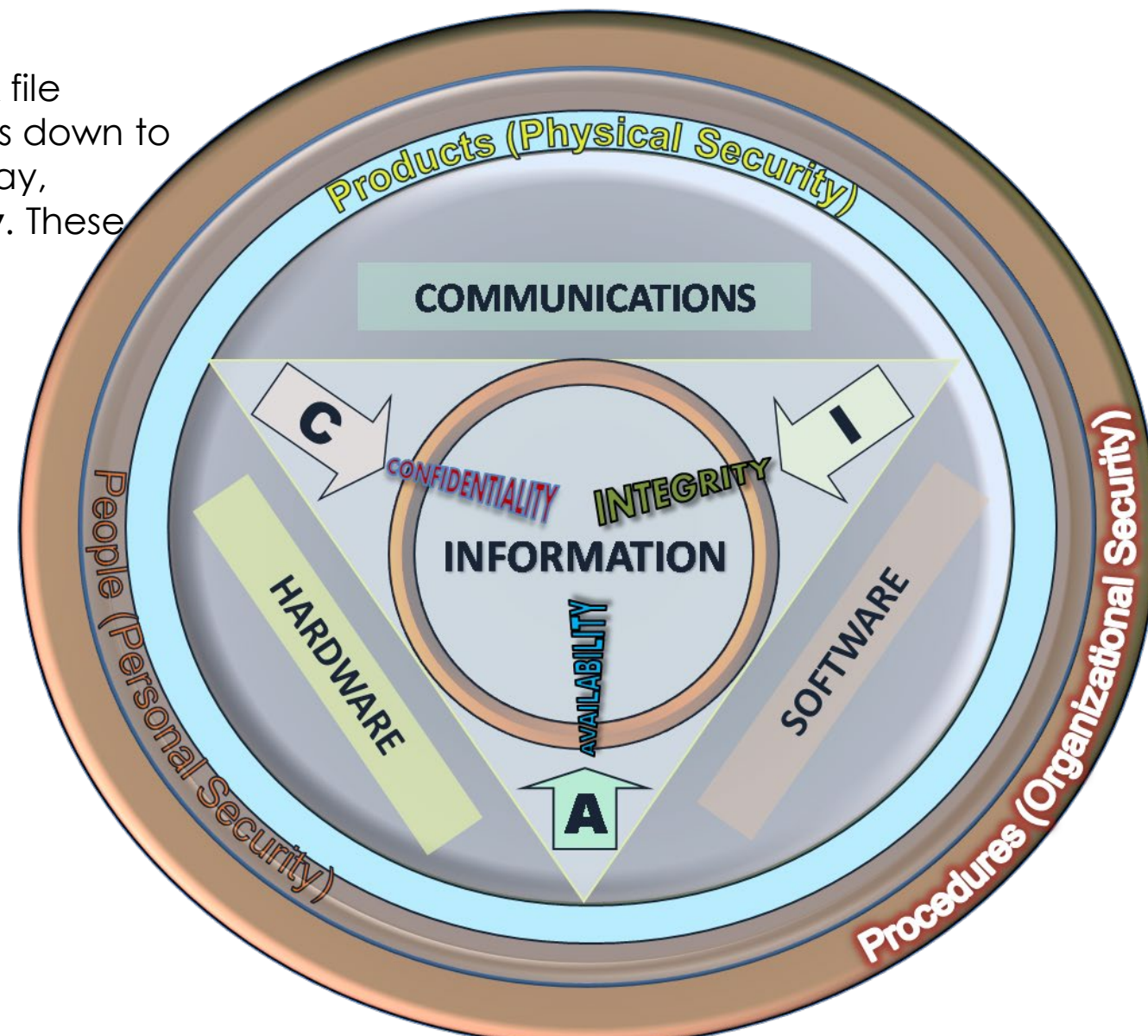
- <https://www.informationisbeautiful.net/visualizations/worlds-biggest-data-breaches-hacks/>

CIA TRIAD

Every security conversation in this field - from a Linux file permission to a billion-dollar compliance audit - boils down to three letters. If you remember nothing else from today, remember CIA: **Confidentiality, Integrity, Availability**. These are the three things a system must protect.

- *A system should provide...*
- **Confidentiality**: only authorized users are able to access data
- **Integrity**: data cannot be manipulated
- **Availability**: data can be accessed when it is needed

What is an attack or technique that hurts these goals?



CLOUD SECURITY

How is security affected by a **distributed system**?

How is security affected by using **the cloud**?

Goal	Example attacks students will offer
Confidentiality	Eavesdropping, data breach, phishing, side-channel attack
Integrity	MITM injection, SQL injection, ransomware (modifies), supply-chain tampering
Availability	DDoS, ransomware (locks out), power outage, fiber cut

CLOUD SECURITY

- How is security affected by a **distributed system**?
 - Almost all software is distributed these days!
 - Distributed systems rely on communication -> open ports are potential weaknesses
 - Many distributed algorithms (like leader election, Raft, etc) can't handle malicious nodes (Byzantine faults)
- How is security affected by using **the cloud**?
 - Many cloud applications are “public facing” -> what if your users are malicious?
 - Clouds share resources for multiple tenants -> what if collocated VMs are malicious?
 - What if the cloud is malicious???